Nib Bank KYC – Complete Beginner-to-Master Guide

This guide is designed to help beginners and advanced users manage the Nib Bank KYC system from GitHub to local setup, database connection, testing, and deployment.

1️⃣ GitHub & Local Setup

1. Install Git: https://git-scm.com/downloads
2. Create GitHub account: https://github.com/
3. Open Command Prompt / Git Bash

Check Git installation:

```
git --version
```

Clone project repository:

```
cd C:\Users\Hp\projects
git clone https://github.com/yourusername/nib-kyc.git
cd nib-kyc
```

Install Node.js dependencies:

```
npm install
```

2️⃣ Configure Environment Variables

Create `.env.local` in project root:

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=YourPostgresPassword
DB_NAME=nib_kyc
NEXT_PUBLIC_FIREBASE_API_KEY=your_firebase_key
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=your_firebase_domain
```

3️⃣ PostgreSQL Local Database Setup

1. Open psql:

```
psql -U postgres
```

2. Create database:

```
CREATE DATABASE nib_kyc;
\c nib_kyc
```

3. Create tables (Users, Posts, Comments, Likes):

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    full_name VARCHAR(150) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    phone VARCHAR(20),
    branch VARCHAR(100),
    role VARCHAR(50),
    password VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE posts (
    id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(id) ON DELETE CASCADE,
    content TEXT NOT NULL,
    image_url TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE comments (
    id SERIAL PRIMARY KEY,
    post_id INT REFERENCES posts(id) ON DELETE CASCADE,
    user_id INT REFERENCES users(id) ON DELETE CASCADE,
    comment TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE likes (
    id SERIAL PRIMARY KEY,
    post_id INT REFERENCES posts(id) ON DELETE CASCADE,
    user_id INT REFERENCES users(id) ON DELETE CASCADE
);
```

4️⃣ Connect Project to PostgreSQL

Create `db.js`:

```
const { Pool } = require('pg');
const pool = new Pool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  password: process.env.DB_PASSWORD,
  port: process.env.DB_PORT,
});
module.exports = pool;
```

Test database connection:

```
const pool = require('./db');
async function testDB() {
    const res = await pool.query('SELECT * FROM users;');
    console.log(res.rows);
}
testDB();
```

5️⃣ Run Locally

```
npm run dev
```

- App URL: `http://localhost:3000` - Test login/signup and CRUD operations

6️⃣ Push Changes to GitHub

```
git add .
git commit -m "Setup local environment and database connection"
git push origin main
```

7️⃣ Deploy to Test Server

1. Use Vercel / Netlify for Next.js:
2. Connect GitHub repo
3. Configure environment variables

4. Auto-deploy on push
5. Use Cloud PostgreSQL (Heroku / AWS RDS)
6. Update `.env` for remote DB credentials
7. Test server URL

8️⃣ Advanced / Master Recommendations

- Role-Based Access Control (Admin, Branch Officer, Employee)
- Activity Logs & Audit Trails
- Notifications for posts/comments
- File uploads (cloud storage with DB references)
- Search, filter, pagination for posts
- API Layer: REST / GraphQL
- JWT authentication & session management
- Secure connections (SSL)
- Backup & restore strategies
- CI/CD pipeline for automation

9️⃣ Nib Bank KYC Flow Core Blueprint

- Verification Workflow Management (Branch Submission, Review Queue, Amendment, Escalation)
- Strategic Hierarchy Approvals (Sequential Sign-off, Governance Nodes, Authorization Memos)
- Head Office Quality Control (Random Sampling, Shared Audit Pool, Compliance Indexing)
- Security & Audit (Domain-Locked Auth, Audit Vault, Zero-Deletion, Master Archiving)
- Advanced Governance Tools (Permission Matrix, KYC F&Q, Institutional Mapping)
- Operational Intelligence (Network Oversight, Officer Productivity, SLA Watchdog)
- Technical Architecture: Next.js 15, Firestore, Firebase Auth, Genkit AI, ShadCN UI
- Regulatory Alignment: Exceeds NBE mandates, secure Nib Bank perimeter

---

🔟 Best Practices

- Never store plain passwords
- Input validation & SQL injection prevention
- Regular backups
- Environment variables for sensitive data
- Maintain clear code structure & modularization
- Hash passwords with bcrypt
- Use feature branches for safe development

✅ This document combines **beginner instructions, local setup, GitHub workflow, PostgreSQL connection, test deployment, and master-level recommendations** for Nib Bank KYC system. It can now be exported as **PDF** for easy reference.